

```
"""Alert and AlertRule definitions for the sensor network data aggregation hub.
```

```
Author: Godspower Istifanus
```

```
This module provides:
```

- * :class:`Alert` -- an immutable-by-convention record describing a single alert raised about a sensor, orderable by severity for use in priority queues.
- * :class:`AlertRule` -- a threshold-based rule that inspects a :class:`~src.readings.SensorReading` and decides whether it warrants raising an :class:`Alert`, and if so, at what severity.

```
"""
```

```
from datetime import datetime
```

```
from typing import Optional
```

```
from src.readings import SensorReading
```

```
class Alert:
```

```
    """Represents a single alert raised about a sensor's behavior.
```

```
    Alerts are ranked by severity so that collections of alerts (e.g. in a priority queue) can be ordered with the most urgent alerts first.
```

```
    Class Attributes:
```

```
        VALID_SEVERITIES: The set of severity levels accepted by the constructor.
```

```
        _SEVERITY_RANK: Maps each valid severity string to a numeric rank used for ordering comparisons. Higher rank means higher urgency (CRITICAL > HIGH > MEDIUM > LOW).
```

```
    Attributes:
```

```
        sensor_id: Identifier of the sensor that triggered the alert.
```

```
        severity: One of "LOW", "MEDIUM", "HIGH", "CRITICAL".
```

```
        message: Human-readable description of why the alert was raised.
```

```
        timestamp: When the alert was generated.
```

```
    """
```

```
    VALID_SEVERITIES = ("LOW", "MEDIUM", "HIGH", "CRITICAL")
```

```
    _SEVERITY_RANK = {
```

```
        "LOW": 0,
```

```
        "MEDIUM": 1,
```

```

        "HIGH": 2,
        "CRITICAL": 3,
    }

def __init__(
    self, sensor_id: str, severity: str, message: str, timestamp: datetime
) -> None:
    """Initialize a new Alert.

    Args:
        sensor_id: Identifier of the sensor that triggered the alert.
        severity: Severity level. Must be one of "LOW", "MEDIUM",
            "HIGH", or "CRITICAL".
        message: Human-readable description of the alert.
        timestamp: When the alert was generated.

    Raises:
        ValueError: If ``severity`` is not one of the valid severity
            levels.
    """
    if severity not in self.VALID_SEVERITIES:
        raise ValueError(
            f"Invalid severity {severity!r}; must be one of "
            f"{self.VALID_SEVERITIES}"
        )

    self.sensor_id = sensor_id
    self.severity = severity
    self.message = message
    self.timestamp = timestamp

def __lt__(self, other: "Alert") -> bool:
    """Compare alerts by severity for use in priority orderings.

    An alert is considered "less than" another if its severity rank is
    lower, so that sorting a list of alerts in ascending order places
    the least urgent alerts first (and descending order / max-heaps
    place CRITICAL first).

    Args:
        other: The other Alert to compare against.

    Returns:
        True if this alert's severity is less urgent than ``other``'s.
    """
    if not isinstance(other, Alert):
        return NotImplemented

```

```

        return (
            self._SEVERITY_RANK[self.severity]
            < self._SEVERITY_RANK[other.severity]
        )

def __str__(self) -> str:
    """Return a concise, human-readable summary of the alert."""
    return f"[{self.severity}] {self.sensor_id}: {self.message}"

def __repr__(self) -> str:
    """Return an unambiguous, developer-facing representation."""
    return (
        f"Alert(sensor_id={self.sensor_id!r}, severity={self.severity!r}, "
        f"message={self.message!r}, timestamp={self.timestamp!r})"
    )

class AlertRule:
    """A threshold-based rule that evaluates sensor readings for a sensor.

    An AlertRule is bound to a specific sensor (by ``sensor_id``) and an
    acceptable value range ``[min_threshold, max_threshold]``. When a
    reading from the matching sensor falls outside that range, the rule
    produces an :class:`Alert` whose severity scales with how far outside
    the range the value is.

    Severity tiers (based on percentage deviation beyond the breached
    threshold):

        * MEDIUM: up to and including 10% beyond the threshold.
        * HIGH: more than 10% and up to and including 25% beyond the
          threshold.
        * CRITICAL: more than 25% beyond the threshold.

    Attributes:
        sensor_id: Identifier of the sensor this rule applies to.
        min_threshold: Minimum acceptable value (inclusive).
        max_threshold: Maximum acceptable value (inclusive).
    """

    def __init__(
        self, sensor_id: str, min_threshold: float, max_threshold: float
    ) -> None:
        """Initialize a new AlertRule.

    Args:
        sensor_id: Identifier of the sensor this rule applies to.
        min_threshold: Minimum acceptable value (inclusive).

```

```

        max_threshold: Maximum acceptable value (inclusive).
    """
    self.sensor_id = sensor_id
    self.min_threshold = min_threshold
    self.max_threshold = max_threshold

def _severity_for_deviation(self, pct_deviation: float) -> str:
    """Determine severity tier for a given percentage deviation.

    Args:
        pct_deviation: How far outside the breached threshold the
            value is, expressed as a fraction (e.g. 0.15 for 15%).
            Always non-negative.

    Returns:
        The severity string corresponding to the deviation tier.
    """
    if pct_deviation <= 0.10:
        return "MEDIUM"
    if pct_deviation <= 0.25:
        return "HIGH"
    return "CRITICAL"

def check(self, reading: SensorReading) -> Optional[Alert]:
    """Evaluate a sensor reading against this rule.

    If ``reading.sensor_id`` does not match this rule's ``sensor_id``,
    the rule does not apply and ``None`` is returned without further
    checks. Otherwise, if the reading's value falls outside
    ``[min_threshold, max_threshold]``, an Alert is created with a
    severity based on the percentage by which the value breaches the
    nearest threshold:
        * within 10% over/under the threshold -> "MEDIUM"
        * more than 10% and up to 25% -> "HIGH"
        * more than 25% -> "CRITICAL"
    If the value is within range, ``None`` is returned.

    Args:
        reading: The SensorReading to evaluate.

    Returns:
        An Alert describing the threshold breach, or None if the rule
        does not apply or the reading is within the acceptable range.
    """
    if reading.sensor_id != self.sensor_id:
        return None

```

```

    if self.min_threshold <= reading.value <= self.max_threshold:
        return None

    if reading.value > self.max_threshold:
        breached_threshold = self.max_threshold
        direction = "above"
    else:
        breached_threshold = self.min_threshold
        direction = "below"

    # Guard against a zero threshold to avoid division by zero; treat
    # any breach of a zero threshold as maximally severe.
    if breached_threshold == 0:
        pct_deviation = float("inf")
    else:
        pct_deviation = abs(reading.value - breached_threshold) / abs(
            breached_threshold
        )

    severity = self._severity_for_deviation(pct_deviation)
    message = (
        f"Reading {reading.value}{reading.unit} is {direction} threshold "
        f"({breached_threshold}{reading.unit}) by {pct_deviation:.1%}"
    )

    return Alert(
        sensor_id=reading.sensor_id,
        severity=severity,
        message=message,
        timestamp=reading.timestamp,
    )

def __str__(self) -> str:
    """Return a concise, human-readable summary of the rule."""
    return (
        f"AlertRule({self.sensor_id}: "
        f"[{self.min_threshold}, {self.max_threshold}])"
    )

def __repr__(self) -> str:
    """Return an unambiguous, developer-facing representation."""
    return (
        f"AlertRule(sensor_id={self.sensor_id!r}, "
        f"min_threshold={self.min_threshold!r}, "
        f"max_threshold={self.max_threshold!r})"
    )

```

